

• Topic : Merge SortQuestion 1:

```
#include <iostream>
using namespace std;
void merge(int arr[], int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
    {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++)
    {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0; j = 0; k = left;
    // Merge
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k++] = L[i++];
        }
        else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) {
        arr[k++] = L[i++];
    }
}
```

```
while (j < n/2)
```

```
{ arr[k++] = R[j++];  
}
```

```
void mergeSort(int arr[], int left, int right)
```

```
{ if (left < right)
```

```
{ int mid = (left + right) / 2;
```

```
mergeSort(arr, left, mid);
```

```
mergeSort(arr, mid+1, right);
```

```
merge(arr, left, mid, right);
```

```
}
```

```
}
```

```
int main( )
```

```
{
```

```
int N;
```

```
cin >> N;
```

```
int arr[N];
```

```
for (int i = 0; i < N; i++)
```

```
{
```

```
cin >> arr[i];
```

```
}
```

```
mergeSort(arr, 0, N-1);
```

```
for (int i = 0; i < N; i++)
```

```
cout << arr[i] << " ";
```

```
return 0;
```

```
}
```

```
PS D:\Weekly modules\Week 3 Module> cd "d:\Weekly modules\Week 3 Module\" ; if ($?) {  
}
```

```
6
```

```
450 1200 800 650 300 1500
```

```
300 450 650 800 1200 1500
```

```
PS D:\Weekly modules\Week 3 Module>
```

Question: 2

By using built-in sort() function

```
#include <iostream>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int N;
```

```
    cin >> N;
```

```
    int arr[N];
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
        cin >> arr[i];
```

```
    }
```

```
    sort(arr, arr+N);
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
// Find median
```

```
    int median;
```

```
    if (N%2 == 0)
```

```
        median = (arr[N/2-1] + arr[N/2]) / 2;
```

```
    else
```

```
        median = arr[N/2];
```

```
    cout << "Median : " << median << endl;
```

```
    int count = 0;
```

```
    for (int i=0; i<N; i++)
```

```
    {
```

```
        if (arr[i] > median)
```

```
            count++;
```

```
    }
```

```

cout << "Orders Above Median: " << count << endl;
cout << "Difference: " << arr[N-1] - arr[0];
return 0;
}

```

- Time Complexity

→ Merge Sort $O(N \log N)$

→ Finding median: $O(1)$

Overall Time Complexity: $O(N \log N)$

- Space Complexity: $O(N)$

```

PS D:\Weekly modules> cd "d:\Weekly modules\Week 3 Module\" ; if ($?) { g++ mergeSort2.cpp -o mergeSort2 } ; if
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28

```


Topic 2: Quick Sort

Question 1:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p-1);
        quickSort(arr, p+1, high);
    }
}

int main()
{
    int n;
    cin >> n;
```

```

int arr[n];
for (int i=0; i<n; i++)
    cin >> arr[i];
quickSort(arr, 0, n-1);
for (int i=0; i<n; i++)
    cout << arr[i] << " ";
return 0;
}

```

- Time Complexity

- Best Case: $O(N \log N)$
- Average Case: $O(N \log N)$
- Worst Case: $O(N^2)$

- Space Complexity

- $O(\log N)$

```

PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ quickSort1.cpp -o quickSort1 } ; if ($?) {
6
120 45 78 200 60 95
45 60 78 95 120 200

```

Question 2:

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int N;
    cin >> N;
    long long arr[N];
    long long sum = 0;
    for (int i = 0; i < N; i++)
    {
        cin >> arr[i];
        sum += arr[i];
    }
    sort(arr, arr + N, greater<long long>());
    for (int i = 0; i < N; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    cout << "Top 5: ";
    long long top5Sum = 0;
    for (int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
        top5Sum += arr[i];
    }
    cout << endl;
    cout << "Average of Top 5: " << top5Sum / 5 << endl;
```

// Overall average

double overallAvg = (double) sum / N;

int count = 0;

for (int i = 0; i < N; i++)

{
 if (arr[i] > overallAvg)
 count++;

}

cout << "Values Above Overall Average:" << count;

return 0;

}

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ quickSort2.cpp -o
8
500 1200 700 2000 900 1500 3000 2500
3000 2500 2000 1500 1200 900 700 500
Top 5: 3000 2500 2000 1500 1200
Average of Top 5: 2040
Values Above Overall Average: 3
PS D:\Weekly modules>
```


Topic 3: Heap Sort

Question 1:

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify(int arr[], int n, int i)
```

```
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
    {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest])
    {
        largest = right;
    }

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
void heapSort(int arr[], int n)
```

```
{
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```

```

int main()
{
    int n;
    cin >> n;
    int arr[n];
    for (int i=0; i<n; i++)
        cin >> arr[i];
    heapSort(arr, n);
    for (int i=0; i<n; i++)
        cout << arr[i] << " ";
    return 0;
}

```

- Time Complexity : $O(N \log N)$
- Space Complexity : $O(1)$ (in-place sorting, ignoring recursion stack)

```

PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ heapSort1.cpp -o heapSort1 } ; if ($?) { .\heapSort1 }
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500

```

Question 2:

```
#include <iostream>
#include <iomanip>
using namespace std;

void heapify (int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap (arr[i], arr[largest]);
        heapify (arr, n, largest);
    }
}

void heapSort (int arr[], int n)
{
    for (int i = n/2 - 1; i >= 0; i--)
        heapify (arr, n, i);

    for (int i = n - 1; i > 0; i--)
    {
        swap (arr[0], arr[i]);
        heapify (arr, i, 0);
    }
}
```



```
int main()
{
```

```
    int n;
    cin >> n;
```

```
    int arr[n];
```

```
    double sum=0;
```

```
    for (int i=0; i<n; i++)
    {
```

```
        cin >> arr[i];
```

```
        sum += arr[i];
```

```
    }
```

```
    heapSort(arr, n);
```

```
    for (int i=0; i<n; i++)
```

```
        cout << arr[i] << " ";
```

```
    cout << endl;
```

```
    cout << "Fastest: " << arr[0] << endl;
```

```
    cout << "Slowest: " << arr[n-1] << endl;
```

```
    double avg = sum / n;
```

```
    cout << "Average: " << fixed << setprecision(2) << avg
    << endl;
```

```
    int count=0;
```

```
    for (int i=0; i<n; i++)
```

```
    { if (arr[i] < avg)
```

```
        count++;
```

```
    }
```

```
    cout << "Cases Faster than Average: " << count << endl;
```

```
    double percentage = (count * 100.0) / n;
```

```
    cout << "Percentage: " << fixed << setprecision(2)
    << percentage << "%";
```

```
    return 0;
```

```
}
```

```
PS D:\Weekly modules> cd "d:\Weekly modules\" ; if ($?) { g++ heapSort2.cpp -o heapSort2 } ; if ($?) { .\
8
12 7 15 9 20 5 10 8
5 7 8 9 10 12 15 20
Fastest: 5
Slowest: 20
Average: 10.75
Cases Faster Than Average: 5
Percentage: 62.50%
```